Module 5 — Laboratory: Test It, Automate It, Ship It

Vistula University · IT Fundamentals · Laboratory assignment

Vistula University · IT Fundamentals

Estimated time: 45–90 minutes

Prerequisites:

- Your **Student Task Manager** project from Module 4 works (tasks can be added, shown, marked done, deleted — with Supabase login).
- Node.js version 18 or newer is installed. Check it: open a terminal and run `node -v`.
- You have a GitHub account and your project is already on GitHub.
- Your project is published on GitHub Pages (from Module 4).

What you will learn today: you will write your first automated tests, add them to your project, and let GitHub test your code automatically after every push. Then you will deploy and check that everything still works.

Task 1 — Plan your tests first (a test table)

Good testers write the test plan **before** the code. We will test a small helper function:

`formatTaskTitle(title)` — takes a task title (text) and returns it cleaned up:

- removes spaces at the start and at the end,
- if the result is empty, returns `null` (which means "no valid title"; the caller then shows an error).

Important idea: this function is a *pure function*. That means it only looks at its input and gives an output — it does not touch the page or the database. Pure functions are the easiest to test.

Copy this table into your notes (or a file `TESTS.md`) and **fill in the "Actual result" column later, in Task 3**:

#	Test name	Input	Steps	Expected result	Actual result	Pass?
1	trims spaces	`" Buy milk "`	call `formatTaskTitle`	`"Buy milk"`	?	?
2	keeps normal text	`"Read a book"`	call `formatTaskTitle`	`"Read a book"`	?	?
3	empty string is rejected	`""`	call `formatTaskTitle`	`null`	?	?
4	only spaces is rejected	`" "`	call `formatTaskTitle`	`null`	?	?
5	(your own idea)	e.g. `"\t\t"` or a very long text	call `formatTaskTitle`	your decision	?	?

Tests 1 and 2 are **positive tests** (valid input). Tests 3 and 4 are **negative tests** (invalid input). Add at least one test idea of your own in row 5.

Task 2 — Implement the function in a separate file

Create a new file `js/utils.js` in your project. Keeping helper functions in a separate file makes them easy to test (and easy to reuse).

```
// js/utils.js
// Small helper functions for the Student Task Manager.

function formatTaskTitle(title) {
  if (typeof title !== "string") {
    return null; // not text at all -> invalid
  }
  const trimmed = title.trim(); // removes spaces at start and end
  if (trimmed.length === 0) {
    return null; // empty or only spaces -> invalid
  }
  return trimmed;
}

// Make the function available in the browser page:
window.formatTaskTitle = formatTaskTitle;

// Make it available for Node tests too (we need this in Task 3):
if (typeof module !== "undefined") {
  module.exports = { formatTaskTitle };
}
```

Now **use it in your project**. In your main file (for example `js/app.js`), find the place where a new task is added, and clean the title before saving:

```
const cleanTitle = formatTaskTitle(titleInput.value);
if (cleanTitle === null) {
  showError("The task title cannot be empty.");
  return; // do not save the task
}
// ...continue: save cleanTitle to Supabase
```

Open your page in the browser and check: adding a task with spaces around the title still works, and an empty title now shows a friendly error instead of saving an empty task.

Task 3 — Write your first automated test

Create the file `js/utils.test.js`. We use Node's built-in test tool `node:test` — nothing to install.

```
// js/utils.test.js
// Run with:  node js/utils.test.js
const { test } = require("node:test");
const assert = require("node:assert");
const { formatTaskTitle } = require("./utils.js");

test("trims spaces from the title", () => {
```

```
  const result = formatTaskTitle("  Buy milk  ");
  assert.equal(result, "Buy milk"); // assertion 1
});

test("keeps normal text unchanged", () => {
  const result = formatTaskTitle("Read a book");
  assert.equal(result, "Read a book"); // assertion 2
});

test("rejects an empty title (negative test)", () => {
  const result = formatTaskTitle("");
  assert.equal(result, null); // assertion 3
});

test("rejects a title with only spaces (negative test)", () => {
  const result = formatTaskTitle("   ");
  assert.equal(result, null); // assertion 4
});
```

What happens here:

- Each `test(...)` is one test case from your table.
- Each `assert.equal(...)` is an **assertion** — a check that says "the result must be this". If it is not, the test fails with a red message.
- Tests 3 and 4 are **negative tests**: wrong input, and we expect the program to handle it politely (`null`).

Run the tests in the terminal, in your project folder:

```
node js/utils.test.js
```

You should see something like ✓ `tests 4` and ✓ `pass 4`. Now go back to your table from Task 1 and fill in the **Actual result** and **Pass?** columns.

Try a bug on purpose (2 minutes): change the expected value in test 1 from `"Buy milk"` to `"Buy milk"` and run again. See the red failure — this is exactly what CI will show your team. Change it back.

Task 4 — Create a GitHub Actions workflow (CI)

Now we tell GitHub: "after every push, run these tests automatically."

Create this file in your project:

`.github/workflows/ci.yml`

```
name: CI
on: push

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
```

```
          node-version: 20
      - run: node js/utils.test.js
```

Reading it line by line:

- `on: push` — the **trigger**: the workflow starts on every push.
- `jobs:` → `test:` — one **job** called "test".
- `runs-on: ubuntu-latest` — the job runs on a fresh Linux machine that GitHub gives us.
- `steps:` — the small actions: download the code (`checkout`), install Node 20, run the tests.
- `uses:` means "use a ready-made action"; `run:` means "run this command".

Task 5 — Build, deploy, and watch it go green

1. Add all new files and push:

```
git add js/utils.js js/utils.test.js .github/workflows/ci.yml TESTS.md git commit -m "Add formatTaskTitle with automated tests and CI" git push
```

2. Open your repository on GitHub → the **Actions** tab. You should see your workflow running (a yellow dot ⏳). After about a minute it becomes a green check ✅. Click on the run to see the steps: checkout → setup-node → run tests.
3. If it is red ❌: click the run, read the error, fix your code locally, and push again. This is the normal CI loop — fixing is learning.
4. Confirm **deployment**: open your GitHub Pages address (Settings → Pages shows the URL, something like `https://yourname.github.io/task-manager/`). Hard-refresh the page with `Ctrl + Shift + R` so the browser loads the new files.
5. Check that the site works: the page opens over **HTTPS** (the address starts with `https://`).
6. Demonstrate **CRUD from Module 4** on the live site, logged in with your Supabase account: - **C**reate — add a new task (also try an empty title: the error message appears and nothing is saved — your negative test, live!). - **R**ead — reload the page; the task is still there. - **U**pdate — mark the task as done. - **D**elete — delete the task and confirm it disappears.
7. One teammate (or a second account) logs in and checks they can see their own tasks — authentication (login) works, and they see only what they are allowed to see (authorization).

Task 6 — Submit

Submit through the course platform on the assignment page (Assignments section): paste your GitHub repository URL / deployed app URL / PR URL as required.

- the link to your **deployed application** (GitHub Pages URL),
- the link to your **repository** on GitHub (make sure the Actions tab shows at least one green run),
- your completed test table (TESTS.md or a photo/scan of your notes).

Expected Deliverables

- `js/utils.js` — the pure function `formatTaskTitle`, used by the main app.
- `js/utils.test.js` — at least 4 tests with 2–3+ assertions (positive and negative), all passing.
- `.github/workflows/ci.yml` — workflow that runs the tests on every push.
- Completed test table with **Expected result** and **Actual result** columns filled in.

- A green ✓ Actions run on GitHub.
- The live site on GitHub Pages with working CRUD.

Submission Instructions

1. Push your final code to GitHub. Check that the latest Actions run is green.
2. On the course platform, on the assignment page (Assignments section), paste: - Repository link: `https://github.com/<your-name>/<your-repo>` - Deployed app link: `https://<your-name>.github.io/<your-repo>/`
3. Add a short comment (2–3 sentences): which test surprised you, and why?
4. Deadline: before the next class. If your Actions run is red at the deadline, submit anyway with a comment explaining what failed and what you would fix next.